FuelWise — Arquitetura da Entrega 3

PUCPR · Engenharia de Software · Arquitetura e Soluções Cloud

Artur Ribeiro de Souza · Cristhian Rezende · Lucas Andriani Theiss · Vinicius da Rosa Padilha

Versão de implementação local, 04/11/2026. Publicação cloud e vídeo pendentes. Referência: documentação original da Entrega 2 (53 páginas), enunciado e prompt da Entrega 3. O código e os resultados registrados em VALIDATION.md prevalecem sobre propostas. Não há URL pública inventada; consultar PUBLIC_URLS.md.

1. Introdução e objetivos

FuelWise centraliza a gestão de combustível e frota de transportadoras pequenas e médias. Gerentes administram motoristas/veículos/viagens e aprovam despesas; motoristas consultam postos e operam suas viagens; administradores aprovam transportadoras, mantêm postos e auditam o sistema.

Esta entrega acrescenta dois microservices, BFF, gateway, microfrontends React, Azure Function HTTP e processamento assíncrono. O fluxo demonstrável é CRUD de postos e viagens, agregação de indicadores e auditoria de eventos. O financeiro legado conserva suas regras e sua execução original; não foi distribuído.

Prioridades: segurança por perfil/transportadora, integridade e idempotência, testabilidade, operação demonstrável sem custos não autorizados. Disponibilidade 99% e p95 até 2 s do PDF continuam metas, não resultados medidos.

Rastreabilidade

RF	Responsável nesta versão
RF01, RF02, RF14	Legado: solicitação, gerente, aprovação/recusa
RF03, RF04	Legado: cadastro mestre de motorista e veículo
RF05, RF06	Mongo: postos e histórico de preços no modo distribuído
RF07	Azure SQL: viagem, exclusividade, estado e desativação
RF08	GPS legado, com mock/cache; não foi migrado para viagens novas
RF09–RF11	Mongo: geolocalização/filiados/detalhes; consulta externa legado preservada no modo original
RF12, RF13	Financeiro legado; indicadores da amostra via Function; auditoria de viagens via eventos
RF15, RF16	Abastecimento/pagamento/denúncia preservados no modo original

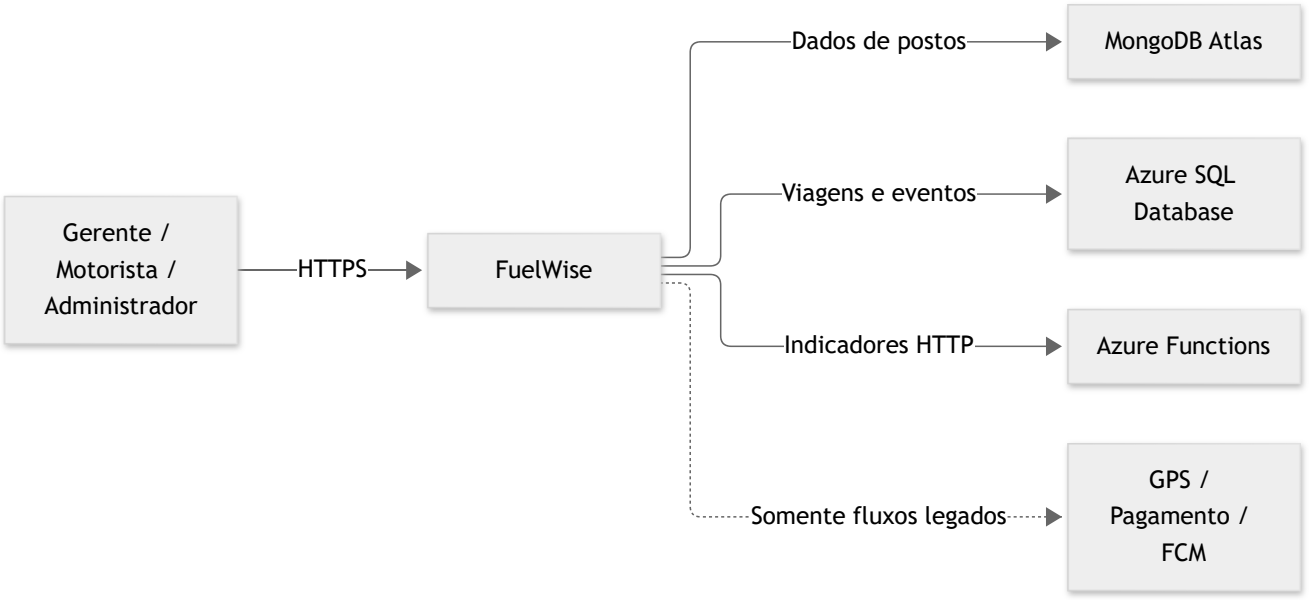
Detalhamento dos contratos anteriores: REQUIREMENTS.md e API.md.

2. Restrições arquiteturais

Dois microservices obrigatórios: MongoDB Atlas Free e Azure SQL Database em oferta gratuita elegível. SQL Server local serve exclusivamente para testes. Node 22/ESM/Express e padrões Vertical Slice/Clean Architecture são mantidos. Frontend não acessa bancos nem serviços diretamente; BFF não possui driver de banco. Nenhum recurso pago, crédito estudantil ou repositório público é criado automaticamente. CLOUD_COSTS.md registra limites e bloqueios de elegibilidade.

O “Free 1 DTU” do enunciado não exige contratar SKU pago: a oferta atual documentada usa vCore-segundos. Quatro microservices do PDF são arquitetura futura e não são implementados nesta entrega.

3. Contexto e escopo (C4 Contexto)



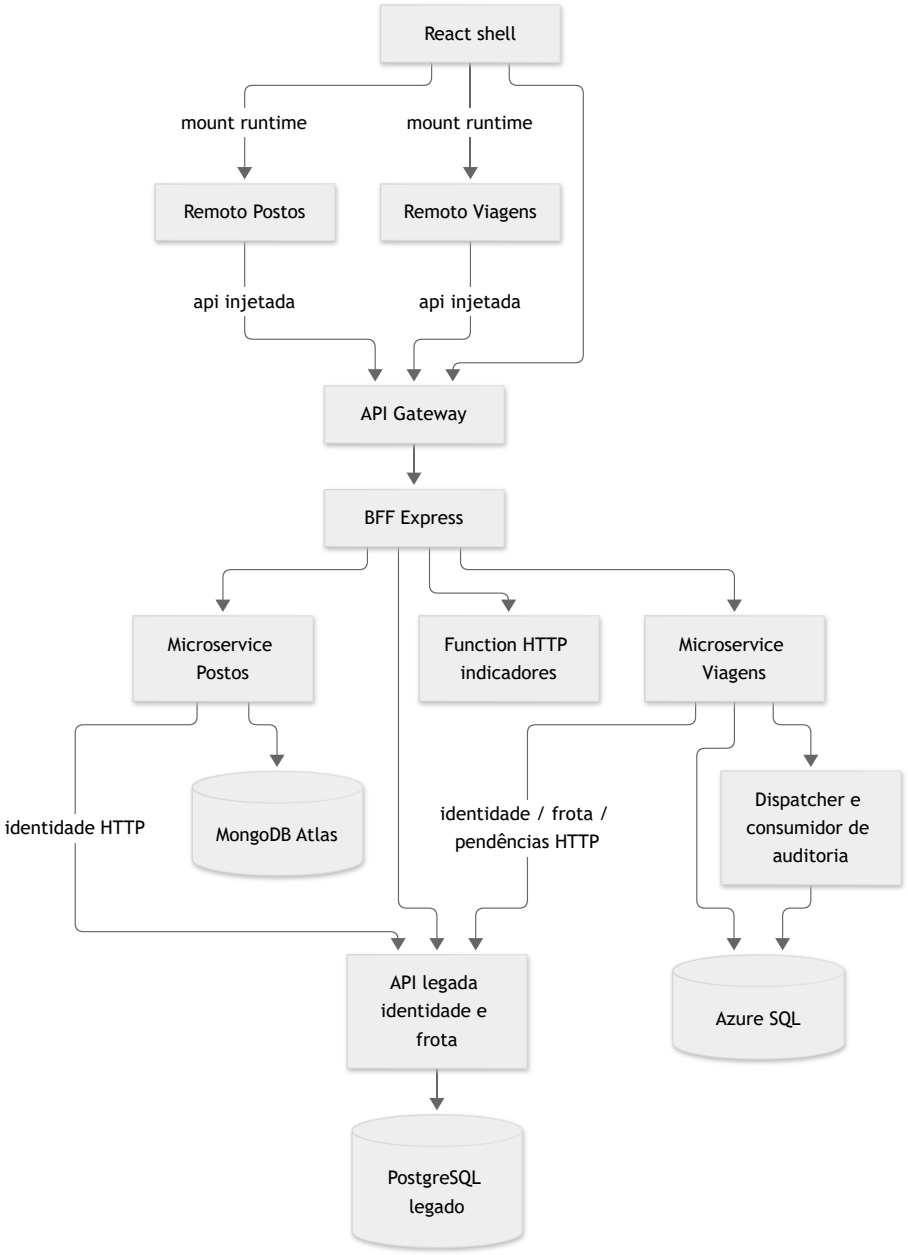
Os provedores externos continuam adaptadores de referência e mocks explícitos na homologação original. O novo fluxo não depende de pagamento comercial.

4. Estratégia de solução

Extrair contratos e regras, manter o repositório original e criar cinco projetos exportáveis. Identity lookup HTTP reutiliza JWT, perfil e estado ativo do legado. Todos os filtros de tenant vêm da identidade validada; não de campos do cliente.

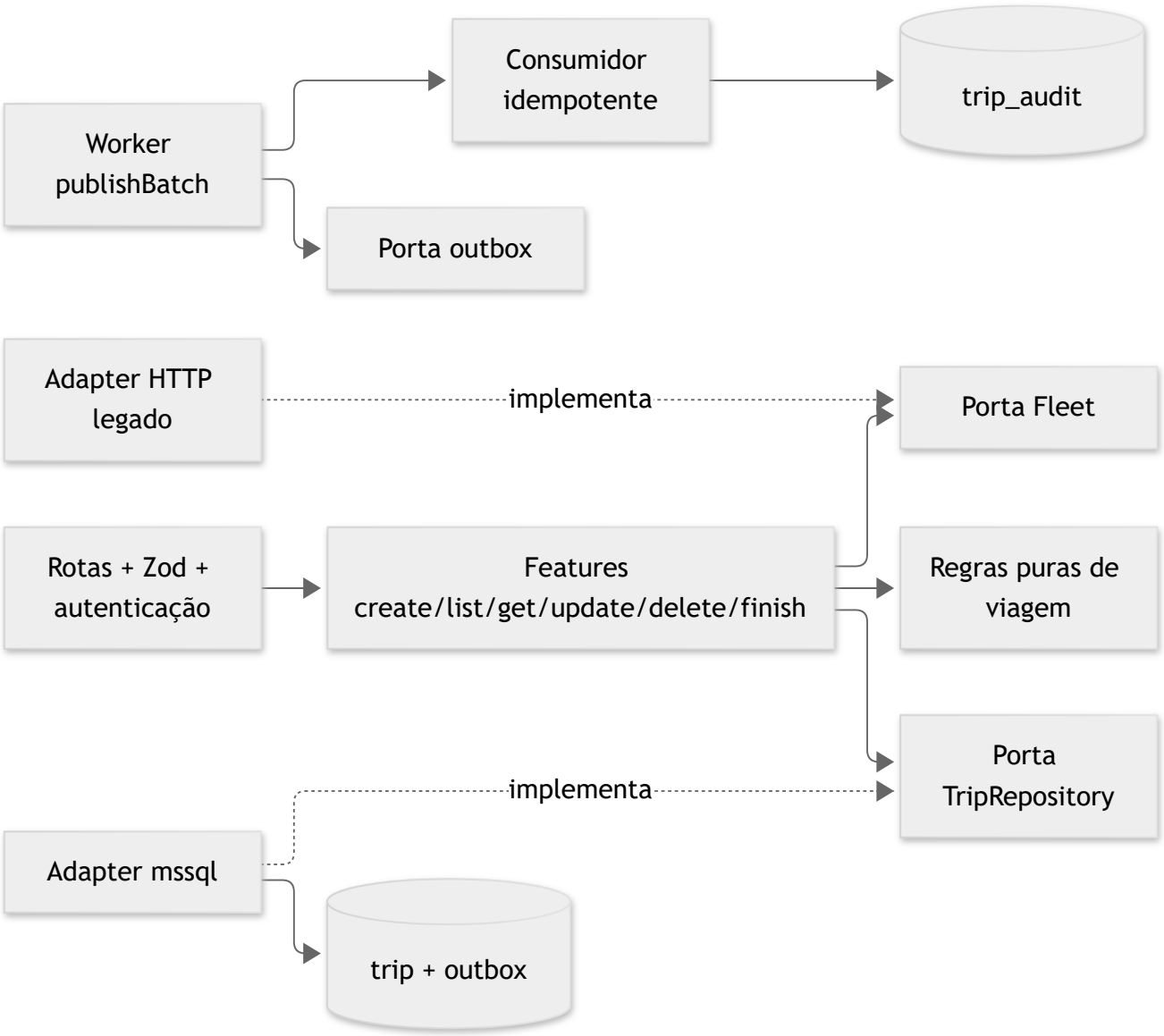
Dois modos de operação: original (todos os slices/PostgreSQL) e distribuído (novos donos de postos e viagens). No distribuído, o legado encaminha as rotas extraídas e recusa inicializar sobre domínios PostgreSQL já preenchidos. Evita uma migração silenciosa ou duas cópias graváveis. A migração de dados e ponte financeira exige trabalho adicional antes de uso com instalações existentes.

5. Blocos de construção (C4 Contêineres)



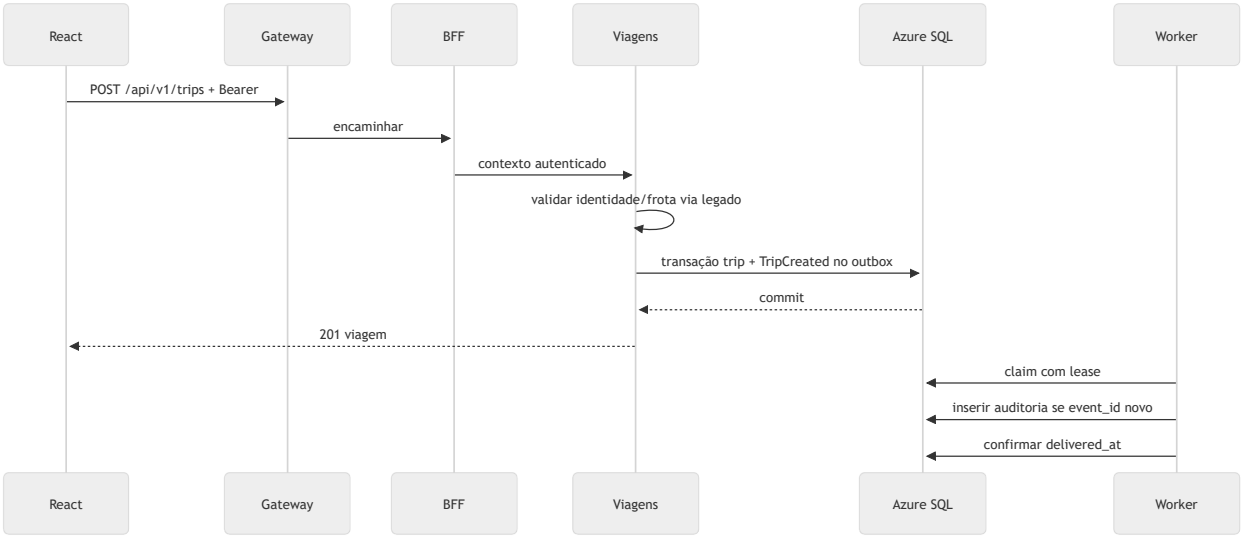
O legado e seu banco são infraestrutura de transição, não um terceiro microservice extraído. As duas fronteiras de negócio novas são Postos e Viagens. Não existe leitura direta de um banco por outro serviço.

C4 Componentes — Viagens



Composition roots constroem adapters. Domain/application/useCase não importam Express ou drivers; testes estáticos verificam dependências e ciclos. Features organizam mudanças por caso de uso; infraestrutura implementa portas pequenas.

6. Tempo de execução



Agregação: BFF consulta Postos e Viagens concorrentemente, envia arrays à Function e retorna indicadores + dados. Até 100 registros por serviço, declarado como amostra. Falha parcial retorna 207, null e mensagens; falha de autenticação retorna 401. Nenhum zero é inventado para um serviço indisponível.

Atualização de viagem permite observações, mantendo motorista/veículo imutáveis. Finalização exige IN_PROGRESS e ausência de pendências; DELETE somente desativa viagem FINISHED. Índices únicos impedem viagens ativas concorrentes. O modo distribuído ainda não aceita abastecimentos novos do financeiro legado, portanto a ponte de concorrência financeira não deve ser considerada homologada.

7. Implantação

Local: Docker Compose isolado, PostgreSQL, Mongo e SQL Server Developer; containers independentes dos componentes. Function usa handler com adapter Node no Compose padrão; overlay opcional validado com host oficial Node 22, chave de função e Azurite local. Deploy Azure é validação separada. Dados persistem em volumes; credenciais em arquivo ignorado. Gateway em localhost:18080 e frontend :5173.

Cloud preparado, não provisionado: Static Web Apps Free, Atlas Free, Azure SQL gratuito elegível, Function com custos comprovados e serviços Docker em hospedagem gratuita revisada. Todas as URLs externas devem usar HTTPS. Ver DEPLOYMENT.md. Cada projeto possui README/package/lockfile; Dockerfiles de BFF/Mongo/SQL usam usuário node e health checks. CI executa testes, build frontend e smoke Compose.

8. Conceitos transversais

- Segurança: Bearer validado pelo legado, RBAC e tenant; sem credenciais no frontend; token em memória; destinos do proxy fixos; CORS explícito.
- Persistência: UUID, parâmetros SQL tipados, pool limitado, TLS validado no Azure; sem exclusão física no CRUD; preços históricos com três casas.

- Eventos: outbox transacional, lease 60 s, entrega pelo menos uma vez, retry exponencial até cinco tentativas, dead letter, deduplicação atômica event_id.
- Observabilidade: logs JSON por componente, health/readiness, auditoria por transportadora e AsyncAPI. Não registrar tokens ou connection strings.
- Resiliência: timeout HTTP, erros padronizados, sem retry automático de CRUD; falhas parciais visíveis. Original financeiro preserva UNKNOWN/idempotência.
- UI: shell e dois ES modules independentes; contrato versionado mount/unmount, API injetada pelo shell, sem URLs de serviço nos remotos.

9. Decisões arquiteturais

Decisão	Motivo / consequência
Dois serviços em vez dos quatro propostos	Escopo obrigatório da Entrega 3; financeiro permanece no legado
Driver Mongo e mssql explícitos	Controle de persistência, queries parametrizadas e teste dos adapters
Identidade HTTP reaproveitada	Conta desativada/transportadora bloqueada têm efeito imediato; dependência do legado
Outbox + worker sem broker pago	Publicação/consumo reais sem Service Bus; operação em uma fronteira de serviço
ES modules com mount/unmount	Deploy independente dos remotos sem infraestrutura de federation; React por raiz
Reverse proxy Node independente	Gateway real gratuito, TLS configurável e limites; rate limit em memória para uma instância
Modo distribuído exige domínios legados vazios	Impede perda/split-brain; migração de produção explicitamente pendente

As decisões de cálculo monetário, aprovação, reserva antes da cobrança, idempotência e conciliação do PDF continuam válidas no legado.

10. Requisitos de qualidade

Cenário	Critério / validação
Token ausente/inválido	401, sem consulta indevida
Perfil não autorizado	403 para mutações de outro perfil
Outra transportadora	404, sem revelar entidade
Segunda viagem ativa	409 garantido por índice SQL
Evento entregue duas vezes	Uma auditoria por event_id
Serviço fora do ar	Timeout e erro/resultado parcial explícito

Cenário	Critério / validação
Import de driver no domínio	CI falha
Remoto atualizado	Build próprio, shell carrega URL configurada

Resultados efetivamente executados estão em VALIDATION.md. Testes locais não atestam SLA, capacidade, segurança de produção ou integração cloud.

11. Riscos e débitos técnicos

Publicação pública, Atlas/Azure reais, implantação Function, vídeo e screenshots cloud pendentes. Gratuidade da Function/storage exige comprovação. Free hosts têm cold start, quotas e suspensão; polling de outbox pode impedir pausa SQL. Rate limit do gateway é por processo. BFF limita dashboard a 100 registros. Migração financeira/denúncias/GPS para novas entidades não implementada; operação original continua separada. Não há rollback distribuído de pagamentos. Produção depende de teste de carga, backup/restauração, revisão de ameaças e observabilidade operacional. Ver PUBLIC_URLS.md, CLOUD_COSTS.md e DEMO_SCRIPT.md.

Software Architecture Canvas — FuelWise Entrega 3

Bloco	Conteúdo
Functional overview	Postos/preços, viagens/frota, indicadores e eventos; identidade e financeiro legado preservados
Business context	Pequenas/médias transportadoras; gerentes, motoristas e administradores; redução de controles manuais
Quality goals	Isolamento, autorização, persistência confiável, idempotência, testabilidade e custo controlado
Organisational constraints	Quatro integrantes, entrega acadêmica, todos falam no vídeo; URLs públicas e documentação rastreável
Technical constraints	React, Node, dois microservices, Atlas Free, Azure SQL, Function HTTP, Gateway, Docker, Clean/Vertical Slice/EDA
External interfaces	Navegador→Gateway→BFF; BFF→serviços/Function/identidade; serviços→seu próprio banco
Architectural hypotheses	Dois domínios extraídos bastam para demonstrar a entrega; ES modules permitem deploy independente; outbox evita broker pago
Solution approach	Incrementos com commits, contratos reaproveitados, portas injetadas, SQL parametrizado, remotos runtime, worker idempotente
Technical challenges and risks	Gratuidade real, cold start, ponte financeira e migração de dados, polling SQL, recuperação de eventos, captura de evidência pública
Decisions and validation	ARC42_DELIVERY3.md, IMPLEMENTATION_PLAN.md, VALIDATION.md; nenhum deploy ou teste inventado

Canvas representa a implementação desta entrega, não a arquitetura futura de quatro serviços do PDF original. Material cloud ainda depende dos gates indicados.

Validação executada — 03 e 04/11/2026

Ambiente

Windows, Node 22.22.0 portátil em `tmp/` (não versionado), Docker Desktop, MongoDB 7 e SQL Server 2022 Developer locais; PostgreSQL 16 para identidade legada. Nenhum banco cloud, recurso pago ou repositório remoto criado.

Resultados

Verificação	Resultado observado
Baseline antes das alterações	37/38 testes; falha apenas no nome do PDF recebido
Legado + arquitetura distribuída	40 testes aprovados (38 anteriores + 2 de fronteiras/ciclos)
Mongo isolado	CRUD HTTP, validação e RBAC aprovados
Mongo real local	Mesmo CRUD executado com driver Mongo e banco temporário: aprovado
SQL isolado	Regras/HTTP, normalização UUID, publicação/ack/retry: 5 aprovados
SQL real local	CRUD, concorrência, isolamento, rollback do outbox e deduplicação: aprovado
BFF	4 testes aprovados: agregação, parcial, 401 e timeout
Function	3 testes aprovados: indicadores, null/vazio e validação
Gateway	Roteamento, CORS e limite: aprovado
Frontend	3 builds separados e teste de fronteiras aprovados
Docker	Imagens construídas e stack local iniciado; health de APIs/bancos/gateway saudável
Smoke distribuído	Identidade, posto/preços CRUD, viagem CRUD/finalização, exclusividade, agregação e consumo TripCreated: PASS
Navegador Edge headless	Login ADMIN/MANAGER, criar posto/viagem, finalizar viagem, dashboard e Swagger BFF; sem pageerror

`node scripts/validate-all.mjs`: 55 testes aprovados, dois testes de banco ignorados no host sem variáveis. Esses dois foram executados separadamente nos containers com bancos reais (Mongo e SQL). Não confundir skip do host com falta de validação local, nem validação local com homologação Atlas/Azure.

Falhas encontradas e corrigidas

- Convenção de nomes aplicada ao documento de entrada: preservado o nome original, com exceção restrita a docs/codex no teste.
- Dependências `file:../..` adicionadas pelo npm --prefix: removidas; lockfiles regenerados no diretório correto e builds independentes executados.
- Porta Windows 8080 indisponível: gateway publicado somente em loopback 18080.
- UUIDs SQL em maiúsculas: comparação canônica, coberta por regressão.
- Ordem de teardown do teste SQL: conexão encerrada depois da limpeza.

Pendências que impedem aceite cloud completo

Atlas Free e Azure SQL gratuitos reais; implantação Azure Functions; HTTPS público; testes de custo/quotas; publicação dos cinco GitHub e três imagens; screenshots cloud; vídeo não listado com os quatro integrantes. CI foi preparado, mas não foi executado no GitHub porque não houve push.

Financeiro/GPS/denúncias de entidades novas não foram migrados: execução original é preservada. Corte sobre banco legado preenchido é deliberadamente bloqueado. Testes de carga, SLA, recuperação de desastre e auditoria de segurança pendentes.

Retomada de 04/11/2026

Estado inicial: Git limpo, HEAD ddf94a8 já contendo documentação/PDF/evidências. Nenhuma alteração pendente perdida. Requisitos e PDF original relidos.

- Host oficial Azure Functions v4 com Node 22, via Dockerfile.runtime: aprovado.
- Azurite local fornece storage sem conta Azure; chaves locais não versionadas.
- HTTP: 401 sem chave e com chave errada; 200 com indicadores; 400 com JSON/status inválidos; null preservado em indisponibilidade parcial.
- Smoke completo pelo Gateway/BFF consumindo esse host: aprovado.
- Zod atualizado para 4.6.7 por GHSA-mrhx-c7xc-m3w5 em quatro projetos. npm audit reportou zero vulnerabilidades nesses projetos após a atualização.
- Regressão: 55 testes passaram; dois testes com banco ignorados no host foram novamente executados com sucesso nos containers Mongo/SQL.
- Checagem sintática: 180 arquivos, zero erros; imagens reconstruídas com sucesso.

CI preparado para repetir host oficial e smoke; execução remota depende de push. Core Tools no Windows não foi executado. O host Docker é runtime real local, mas não substitui Atlas, Azure SQL ou Function hospedados exigidos no aceite final.

Custos e restrição de provisionamento

Consulta às fontes oficiais em 03/11/2026. **Nenhum recurso cloud foi criado.** Créditos Azure for Students não são orçamento autorizado. A existência de uma franquia não comprova custo total zero nem elegibilidade da assinatura pessoal.

Componente	Plano/limites consultados	Uso esperado na demonstração	Risco e proteção	Desativação
Postos	MongoDB Atlas Free, um cluster por projeto; limite de armazenamento deve ser conferido no portal	Menos de 100 postos/preços; pool 5	Selecionar Free explicitamente, sem upgrade, sem extras pagos; usuário restrito ao banco, IPs de saída autorizados	Encerrar cluster após exportar dados, mediante confirmação
Viagens	Azure SQL General Purpose serverless com free limit: 100.000 vCore-s/mês, 32 GB dados e 32 GB backup por banco; até 10 bancos	Menos de 100 viagens; sessões curtas	Usar oferta gratuita e AutoPause ao esgotar; nunca BillOverUsage. Pool 3; desligar worker/containers fora das sessões	Parar aplicações e conexões; manter AutoPause; excluir somente após backup/autorização
Function	Flex Consumption: franquia anunciada de 250.000 execuções/100.000 GB-s; condições de assinatura aplicáveis	Até 1.000 chamadas curtas	Storage/rede não incluídos na franquia. Zero Always Ready, sem Premium; não provisionar até comprovar todos os custos	Parar Function App; revisar storage/logs separadamente
Frontend	Static Web Apps Free: 100 GB banda/assinatura, 0,50 GB storage e 0,25 GB por deploy	Build abaixo de 2 MB; menos de 1 GB transferência	Selecionar Free, sem Standard/Front Door. Limite excedido interrompe site	Excluir app ou desconectar deploy
BFF, serviços, gateway e legado	Candidato: Render Free Web Services, 750 horas/mês compartilhadas no workspace	Cinco processos por 20 horas = cerca de 100 horas	Cold start; banda/build podem gerar excedentes com método de pagamento. Sem cartão e sem upgrade; conferir limites da conta, suspender fora da demonstração	Suspender serviços no painel
Identidade legada	Candidato: Render Free Postgres, 1 GB, expira em 30 dias	Menos de 100 usuários/veículos	Usado somente pelo legado, nunca como substituto de Azure SQL. Backup antes de expirar	Exportar, depois remover com autorização

Componente	Plano/limites consultados	Uso esperado na demonstração	Risco e proteção	Desativação
Eventos	Outbox/auditoria no SQL existente	Poucos eventos por sessão	Polling mantém banco ativo; não executar permanentemente. Sem Service Bus pago	Parar serviço de viagens
Docker Hub/GitHub	Repositórios públicos e imagens preparados, não publicados	Três imagens obrigatórias, cinco projetos	Publicação requer revisão de segredos e autorização; limites da conta precisam ser conferidos	Remover/despublicar apenas com autorização

O enunciado usa “Free 1 DTU”; a oferta atual consultada usa **vCore-segundos**. Não selecionar SKU pago para reproduzir o texto antigo. A FAQ distingue Azure for Students Starter (incompatível) de outras ofertas estudantis. Confirmar o tipo exato da assinatura e a opção gratuita no portal, sem presumir elegibilidade.

A documentação de Functions informa que storage e rede são cobrados à parte e que franquias têm condições específicas. Portanto a implantação de Function permanece **bloqueada por comprovação de custo zero**, mesmo com baixo tráfego. O adaptador Node local não satisfaz a demonstração de Azure Functions na nuvem.

Evidência necessária antes de criar recursos

1. Tipo da assinatura e tela de elegibilidade da oferta SQL.
2. Estimativa zero, free limit e AutoPause habilitados; sem extras.
3. Function/Storage/rede/logs com gratuidade comprovada para a assinatura.
4. Quotas e método de pagamento dos provedores de hospedagem.
5. Consentimento separado se qualquer consumo de crédito/cobrança for necessário.

Fontes

- Azure SQL Free FAQ
- Configuração da oferta SQL
- Azure Functions pricing
- Static Web Apps pricing
- Atlas Free limits
- Render Free

Execução e publicação da Entrega 3

Local, preservando o ambiente original

O `compose.yaml` original continua executando todos os fluxos legados. `compose.delivery.yaml` usa projeto/volumes próprios, sem remover dados antigos. Não executar `down -v` em um ambiente com dados que se deseja preservar.

```
Copy-Item .env.delivery.example .env.delivery
# Preencher senhas fortes e JWT_SECRET, sem versionar .env.delivery.
docker compose --env-file .env.delivery -f compose.delivery.yaml up -d --build
node scripts/smoke-delivery.mjs
```

Frontend: http://localhost:5173; gateway: http://localhost:18080; Swagger BFF: http://localhost:18080/docs; legado: http://localhost:3000. Swagger dos serviços é acessível na rede Docker em stations:3001/docs e trips:3002/docs. Para inspeção local, encaminhar uma porta loopback explicitamente. Os bancos não expõem portas no host. Health checks dos serviços verificam seus bancos. O Compose padrão usa o adaptador Function Node; o overlay abaixo executa o host oficial. A apresentação final continua exigindo Azure Functions público.

Host oficial da Function, sem recursos Azure

```
node scripts/prepare-function-runtime.mjs
docker compose --env-file .env.delivery --env-file .env.function-runtime -f
compose.delivery.yaml -f compose.functions-runtime.yaml up -d --build
node scripts/test-function-runtime.mjs
node scripts/smoke-delivery.mjs
```

O overlay usa a imagem oficial `mcr.microsoft.com/azure-functions/node:4-node22` e Azurite como armazenamento emulado local. A porta 7071 aceita somente loopback; o BFF envia a chave em header. O teste verifica autenticação do host, resultados e entradas inválidas. Os arquivos `.env.function-runtime` e `tmp/function-secrets/` são ignorados. Gerar novas chaves exige recriar Function/BFF/Azurite com Compose. Não publicar esse overlay: suas configurações são exclusivas de desenvolvimento. Core Tools continua disponível como alternativa via `npm start` no projeto da Function. Este teste local não comprova gratuidade nem implantação na Azure.

Para inspecionar os Swagger, adicionar `-f compose.inspect.yaml` ao comando Compose: http://localhost:13001/docs e http://localhost:13002/docs. Apenas portas loopback dos serviços são abertas; o frontend continua usando o gateway. Ao parar a variante oficial, usar os mesmos `--env-file` e `-f` com `down`, sem `-v`.

Referência da imagem: Microsoft - containers Functions.

O teste smoke cria uma transportadora fictícia aprovada, gerente, motorista, veículo, posto/preço e viagem. Verifica CRUD, conflito de viagem ativa, agregação e consumo de evento. Não contém credenciais fixas. Pode ser executado novamente com dados novos. Uma sessão de teste é salva em `tmp/`, ignorada pelo Git.

Parar sem apagar volumes: `docker compose --env-file .env.delivery -f compose.delivery.yaml down`.

Limite da migração

DOMAIN_MODE=distributed intercepta as rotas de postos/viagens do legado e as encaminha aos novos donos. A inicialização recusa esse modo se houver postos ou viagens no PostgreSQL: **não migra nem apaga dados automaticamente**. O modo original continua íntegro. O financeiro antigo permanece disponível no modo original; referências financeiras para viagens novas distribuídas ainda não foram migradas. O fluxo de aceite desta entrega cobre postos, viagens, indicadores e eventos. Não anunciar pagamento distribuído como concluído.

Antes de migrar uma instalação com dados reais: exportar com IDs preservados, conciliar viagens em andamento/abastecimentos, desenhar a ponte financeira com controle de concorrência, congelar os escritores antigos e testar recuperação. O bloqueio atual evita duas fontes mutáveis de verdade por acidente.

Cloud, somente após revisão de CLOUD_COSTS.md

1. Obter Atlas Free e Azure SQL gratuito elegível. Configurar credenciais com privilégio mínimo, IPs de saída e TLS. Executar migrations SQL aditivas.
2. Publicar os serviços a partir de seus diretórios independentes (Dockerfile ou `npm ci`/`npm start`). Injetar variáveis dos `.env.example` em secret settings.
3. Hospedar identidade legada com PostgreSQL próprio. Não trocar o banco de viagens por este PostgreSQL. Usar ambiente de demonstração isolado.
4. Publicar Function em recurso previamente aprovado: `func azure functionapp publish NOME_APROVADO`; configurar FUNCTION_URL HTTPS e FUNCTION_KEY no BFF.
5. Publicar gateway com terminador TLS confiável, configurar BFF_URL e CORS_ORIGIN. Não divulgar as portas internas nem configurar CORS `*`.
6. Publicar `dist` frontend em Static Web Apps Free; editar runtime-config.js com URLs reais HTTPS. Remotos separados exigem CORS da hospedagem estática.
7. Aquecer serviços Free antes da demonstração para lidar com cold start. Não usar pings permanentes que consomem horas/franquias. Parar worker após sessão.
8. Executar smoke contra URLs reais, comprovar Atlas/Azure e coletar evidências.

Docker Hub e cinco repositórios

Somente após autorização, executar `docker login` interativamente; nunca fornecer token na linha de comando ou arquivo versionado. `scripts/docker-images.ps1 -DockerHubUser USUARIO` faz builds; `-Push` publica explicitamente.

`scripts/export-projects.ps1` exporta apenas arquivos rastreados dos cinco projetos, sem node_modules, segredos ou artefatos temporários. Para preservar histórico, alternativa revisável: `git subtree split --prefix=projects/NOME -b export-NOME` e push para repositório autorizado. Não executado automaticamente. O gateway permanece na infraestrutura do repositório original.

Validação

`npm ci` em cada diretório; `node scripts/validate-all.mjs`; `npm run check`; `npm run build` no frontend; Compose e smoke. Integrações cloud são um gate separado e não podem ser substituídas pelos testes isolados.

Links da entrega — preencher somente após publicação real

Artefato	URL / estado
GitHub original	https://github.com/LucasTheiss/fuelwise-architecture (referência fornecida; alterações locais ainda não enviadas)
GitHub frontend	Pendente de autorização/publicação
GitHub BFF	Pendente de autorização/publicação
GitHub Mongo	Pendente de autorização/publicação
GitHub SQL	Pendente de autorização/publicação
GitHub Function	Pendente de autorização/publicação
Docker Hub BFF/Mongo/SQL	Pendente de autorização/publicação
Frontend / Gateway HTTPS	Pendente
Swagger BFF / Mongo / SQL	Pendente
Function HTTP	Pendente de comprovação de custos e deploy
AsyncAPI	Local: docs/asyncapi.json; URL pública pendente
YouTube não listado	Pendente de gravação pelos quatro integrantes

Roteiro — vídeo não listado, quatro integrantes

Usar URLs **públicas HTTPS**, jamais apresentar localhost como entrega cloud. Pré-condições: preencher PUBLIC_URLS.md, confirmar custos, smoke cloud aprovado, contas ADMIN/MANAGER/DRIVER e frota de demonstração. Não mostrar senhas/tokens.

Tempo	Integrante	Demonstração e fala
0–2 min	Artur Ribeiro de Souza	Identificar-se; apresentar objetivo FuelWise e C4; abrir frontend público; mostrar shell e dois remotos no Network; explicar gateway/BFF
2–4 min	Cristhian Rezende	Identificar-se; login ADMIN; cadastrar, consultar, editar e desativar posto; mostrar persistência no Atlas e Swagger Mongo; cadastrar outro posto para dashboard
4–6 min	Lucas Andriani Theiss	Identificar-se; login MANAGER; criar viagem com frota válida, consultar/editar/finalizar/desativar; mostrar SQL Azure, índices e conflito de segunda viagem ativa; Swagger SQL
6–8 min	Vinicius da Rosa Padilha	Identificar-se; chamar /aggregated-data pelo gateway; mostrar logs BFF/serviços/Function e resultado no dashboard; criar evento e mostrar outbox/auditoria; explicar idempotência, testes e limitações

Mostrar Swagger BFF, gateway HTTPS, endpoint real da Function com chave oculta, e rastreio por horário/ID da viagem. Um registro no outbox não basta: mostrar `delivered_at`, auditoria e log `event_consumed`. Reentrega deve manter um efeito. Finalizar com GitHub, Docker Hub e documentação. Não dizer que integrações de pagamento simuladas movimentam dinheiro, nem que o financeiro foi distribuído. Não publicar vídeo automaticamente. Copiar URL não listada para PUBLIC_URLS.md.